

The Offensive Security Master Roadmap

Pentest → Reverse Engineering → Exploit Development → AI Red Teaming — one connected path

This is the deep version of your offensive path. Everything branches off **one shared trunk** (C → assembly → reverse engineering). Build the trunk once, and every specialization below becomes reachable. Read top to bottom; don't jump ahead of the trunk.

Three rules that matter more than any resource: (1) hands-on beats reading — for every concept, go break something; (2) demonstrate publicly — CVEs, writeups, CTF, bug bounty are what get you hired, not finished courses; (3) hold it loosely — specific techniques change, so keep the foundation strong and keep learning. Realistic horizon to be strong across the whole map: **2–4 years** of steady work. That's normal for elite skill.

Legend: [FREE] free [PAID] paid ■ = time estimate (steady ~1–2 hrs/day)

TRUNK · Shared Foundation (do this first, for everything)

Both exploit dev and AI-infrastructure attacks rest on this. No skipping.

1. C programming

■ [CS50x \(Harvard\)](#) [FREE] weeks 0–5 + [K.N. King: C: A Modern Approach](#) as reference.

■ **6–10 weeks. Done when you use pointers, structs & malloc without copying.**

2. Assembly (x86/x64)

■ [Programming from the Ground Up](#) [FREE] + [OpenSecurityTraining2](#) [FREE] (Architecture 1001/2001).

■ **3–6 weeks. Done when you can read disassembly and follow the flow.**

3. Reverse engineering fundamentals

■ [Ghidra](#) [FREE] + [Practical Reverse Engineering](#) (book) + practice on [crackmes.one](#) [FREE] and [FLARE-On challenges](#) [FREE].

■ Video spine: [LiveOverflow](#) [FREE].

■ **4–8 weeks. Done when you can reverse a small crackme unaided.**

TRACK A · Exploit Development

The offensive deep-dive that extends your pentest/bug-bounty work. Start with Linux userland binary exploitation — it is the gateway every other arena builds on. Don't start the advanced arenas until the core feels natural.

A1. Binary exploitation core (START HERE)

Bug classes you'll learn in order: stack buffer overflow → shellcode → ret2win / ret2libc → ROP → format string → heap. Plus defeating mitigations (NX/DEP, ASLR, stack canaries, PIE).

■ [pwn.college](#) [FREE] — the best free curriculum (ASU/Shellphish): video + challenges + tooling from day one. Your main spine.

■ [Nightmare by guyinatuxedo](#) [FREE] — CTF-based course; pair with pwn.college to drill.

■ [ROP Emporium](#) [FREE] — dedicated ROP practice. [exploit.education](#) [FREE] — graded VM challenges.

■ [irOnstone notes](#) [FREE] & [CryptoCat Binexp 101 \(YouTube\)](#) [FREE].

■ Tooling: [pwntools](#) [FREE] (Python exploit library — learn early).

■ Books: *Hacking: The Art of Exploitation* (Erickson) · *The Shellcoder's Handbook*.

■ More practice: [pwnable.kr](#) [FREE], [pwnable.tw](#) [FREE].

■ **3–6 months. Done when you can ROP past NX without a tutorial. Languages: C/asm + Python.**

A2. Windows exploit development

- [Corelan tutorials](#) [FREE] — the classic Windows exploit-writing series (SEH overflows, egghunters).
- [OffSec OSED / EXP-301](#) [PAID] — the recognized Windows exploit-dev cert.
- 2–3 months. Languages: C/asm + Python. Builds on A1.

A3. Heap exploitation

- [how2heap \(shellphish\)](#) [FREE] — the canonical heap technique playground; Nightmare also covers heap.
- 2–3 months. Harder than stack; very relevant to modern targets.

A4. Kernel exploitation

- Linux: [xairy/linux-kernel-exploitation](#) [FREE] + pwn.college kernel modules.
- Windows: [HackSys Extreme Vulnerable Driver \(HEVD\)](#) [FREE]. Book: *A Guide to Kernel Exploitation*.
- 3–6 months. Needs OS internals. Higher privilege, higher difficulty.

A5. Browser exploitation (advanced)

- [m1ghtym0/browser-pwn](#) [FREE] curated path + [Google Project Zero blog](#) [FREE] (real-world writeups).
- 6–12+ months. Among the hardest. Adds JavaScript + heavy C++ internals.

A6. Mobile exploitation

- [OWASP MASTG](#) [FREE] (Android+iOS standard). Android adds Java/Kotlin + Smali; iOS adds Swift/Obj-C; both add ARM asm.
- 3–6 months. The arena that genuinely adds new languages.

A7. Embedded / IoT / firmware

- Book: *Practical IoT Hacking* (No Starch). Tools: binwalk, Ghidra. C-heavy + ARM/MIPS asm.
- 3–6 months.

A8. Fuzzing & vulnerability research (cross-cutting — the bug-finding half)

- [AFL++](#) [FREE] + [The Fuzzing Book \(free online\)](#) [FREE] + [Google fuzzing guide](#) [FREE].
- 2–3 months to get productive; then ongoing. This is how you find NEW bugs (0-days).

TRACK B · AI Security (how it connects)

AI security splits in two. One half reuses your exploit-dev skills directly; the other is a newer, lighter, Python-based specialty. Both extend offensive work, not replace it.

B1. Attacking AI infrastructure (uses Track A directly)

AI systems run on normal software — servers, APIs, C/C++ libraries, MCP servers. These have classic bugs (memory corruption, SSRF, RCE). Real 2026 examples: CVE-tracked exploit chains in MCP servers. **Your C/asm/exploit-dev skills apply straight here — no new core language.**

B2. AI / LLM red teaming (the new specialty)

Attacking the model's behavior through language and data: prompt injection, jailbreaks, system-prompt leakage, agent goal hijacking, RAG poisoning. The 'exploit' is often crafted text. Languages: **natural language + Python**, plus understanding how models/agents are wired. Maps cleanly onto your web/API instincts.

- [OWASP Top 10 for LLM & Agentic Apps](#) [FREE] — the canonical vuln taxonomy (LLM 2025 + Agentic ASI 2026).
- [PortSwigger LLM attack labs](#) [FREE] — hands-on, same platform you know from web. Best starting point.
- [Lakera Gandalf](#) [FREE] — gamified prompt-injection practice.
- [LLM Security Guide \(GitHub\)](#) [FREE] + [Arcanum AI-sec labs hub](#) [FREE] (many labs in one place).
- Tooling: [Garak \(NVIDIA\)](#) [FREE], [PyRIT \(Microsoft\)](#) [FREE], [promptfoo](#) [FREE].

■ Structured course/cert: [Learn Prompting AI Red Teaming \(AIRTP+\)](#) [PAID]. Bug bounty: [Mozilla 0Din GenAI](#) [FREE].

■ 1–3 months to competence (much lighter than Track A). High current demand.

TRACK C · Offensive Malware Development (red-team tooling)

Building custom implants, payloads, and evasion for **authorized** red-team engagements — the offensive cousin of malware *analysis* (which you parked). Prerequisite is RE + C + Windows internals, which the trunk and Track A already build. Some analyst knowledge helps you evade detection, but it's a force-multiplier here, not a hard gate.

C1. Windows internals (the real prerequisite)

■ *Windows Internals* (Rusinovich) — reference for processes, memory, the Win32 API, PE format. Dip in as needed; you'll have met PE files during RE.

C2. Core implant development & injection

■ [MalDev Academy](#) [PAID] — the modern, structured course for Windows malware/implant dev and evasion. The main spine here.

■ [Sektor7 Institute](#) [PAID] — "Red Team Operator: Malware Development" courses (essentials → advanced).

■ [ired.team](#) [FREE] — excellent free notes on process injection, persistence, and offensive Windows techniques.

C3. Evasion & modern tradecraft

■ *Evasive Malware* (Kyle Cucci, 2024) — anti-analysis & EDR evasion, current. Note: modern implants increasingly use C#, Nim, Go, or Rust for evasion — C remains the base to learn first.

■ [OffSec OSEP / PEN-300](#) [PAID] — "Evasion Techniques and Breaching Defenses"; the recognized cert for this skill.

C4. Command & control (C2)

■ [Sliver](#) [FREE], Mythic, Havoc — the frameworks that deliver and manage implants on engagements.

■ 4–8 months bit-by-bit, after RE is solid. Languages: C (base) + C#/Nim/Go/Rust for evasion + Python tooling.

Reality check: this is genuinely advanced and detection moves fast — what evades EDR today may not next year. It's a 'keep learning forever' lane, slotted in during the red-team stage, not before exploit-dev basics.

THE WEB · How it all chains to your pentest career

These aren't separate careers — they feed each other. Here's how the pieces connect into one offensive operator profile:

■ **Web & API** (your base) → [PortSwigger Web Security Academy](#) [FREE] + [OWASP API Top 10](#) [FREE]. Feeds directly into B2 (LLM apps are web apps).

■ **Active Directory / internal** → [GOAD lab](#) [FREE] + [CRTTP \(Altered Security\)](#) [PAID]. The bridge from web pentest to full red teaming.

■ **Endpoint security / EDR evasion** → this is where malware-dev meets offense: [MalDev Academy](#) [PAID]. Your C + RE foundation powers it.

■ **C2 & red team ops** → [Sliver](#) [FREE], Mythic, Cobalt Strike. Where exploit dev + AD + evasion combine on real engagements.

■ **Practice platforms** tying it together: [Hack The Box](#), [TryHackMe](#) — full attack chains end to end.

The mental model: Pentest finds the door → exploit dev builds the key when none exists → AD/endpoint/C2 move through the building → AI red teaming covers the newest doors (models & agents). Same operator, widening reach.

FOLLOW · People, Blogs, GitHub, Video & Communities (by domain)

Books are listed in each track above. Below is who to **follow and read** per domain — videos, blogs, GitHub, and the people doing the work. X/Twitter handles change; if one's off, search the name.

Foundations (C / assembly)

- Video: [freeCodeCamp](#), [Jacob Sorber](#), [Low Level](#), [Neso Academy](#).
- Labs: [OpenSecurityTraining2](#) (free university-grade architecture courses).

Reverse engineering

- Video: [LiveOverflow](#), [OALabs](#), [stacksmashing](#), [LaurieWired](#), [MalwareAnalysisForHedgehogs](#).
- Blogs: [OALabs blog](#) · people: Maria Markstedter / Azeria (ARM), LaurieWired.

Binary exploitation & exploit development

- Video: [CryptoCat](#), [LiveOverflow](#), [PwnFunction](#).
- Blogs/podcast: [dayzerosec](#) (weekly VR/exploit podcast), [Google Project Zero](#), [Zero Day Initiative](#), [Exploit Reversing \(A. Borges\)](#), [Zero Day Engineering \(Alisa Esage\)](#).
- GitHub/notes: [ir0nstone](#), [Nightmare](#), [how2heap](#).
- People: Yan Shoshitaishvili / Zardus (pwn.college), guyinatuxedo, ir0nstone, CryptoCat.

Windows / kernel exploit dev

- Blogs: [Connor McGarr](#) (Windows exploit dev), [Corelan](#).
- People: Alex Ionescu, Yarden Shafir, Pavel Yosifovich (Windows internals).
- Labs: [HEVD](#), [xaivy kernel exploitation](#).

Browser / advanced

- [Google Project Zero](#) (the gold standard), [browser-pwn curated path](#). People: Samuel Gross (saelo).

Offensive malware dev & red team tradecraft

- Blogs: [ired.team \(spothplanet\)](#), [XPN / Adam Chester](#), [SpecterOps](#), [Outflank](#).
- Courses: [MalDev Academy](#), [Sektor7](#). Community archive: [vx-underground](#).
- People: Adam Chester (XPN), Mantvydas Baranauskas (ired.team), reenz0h (Sektor7).

Active Directory / red team ops

- Video: [lppSec](#) (HTB walkthroughs). Writeups: [0xdf](#).
- Blogs: [SpecterOps](#) (BloodHound/AD). People: Will Schroeder (harmj0y), the SpecterOps team.
- Lab: [GOAD](#).

Web & API / bug bounty

- Video: [NahamSec](#), [InsiderPhD](#) (API), [PwnFunction](#).
- Blogs: [PortSwigger Research](#) (James Kettle / albinowax). People: Jason Haddix, STOK, NahamSec, Katie Paxton-Fear.

AI / LLM red teaming

- Blogs: [Embrace The Red \(Johann Rehberger\)](#) (top prompt-injection research), [Simon Willison](#) (clear, frequent writeups).
- Hubs: [OWASP GenAI](#), [Arcanum AI-sec resources](#), [LLM Security Guide](#).
- People: Johann Rehberger (wunderwuzzi), Simon Willison, Sander Schulhoff (HackAPrompt), Leon Derczynski (garak).

Aggregators & communities (to stay current)

- Newsletters/news: [tl;dr sec](#) (Clint Gibler), [dayzerosec weekly](#), [BleepingComputer](#), [The Hacker News](#).
- Reddit: [r/netsec](#), [r/ExploitDev](#), [r/ReverseEngineering](#), [r/AskNetsec](#).

- CTF + practice: [CTFtime](#), [Hack The Box](#), [TryHackMe](#). Discords: pwn.college, MalDev Academy, HTB.
- Curated lists: search GitHub for [awesome-pentest](#), awesome-malware-analysis, awesome-llm-security.
- X/Twitter: follow the people named above — the infosec community there is where 0-days, writeups, and techniques break first.

MILESTONES · Optional certs that mark progress

- **OSCP** (PEN-200) — the offensive baseline; do this off your pentest base.
- **CRTP / CRT0** — Active Directory / red team operations.
- **OSWE** (WEB-300) — advanced web exploitation.
- **OSED** (EXP-301) — Windows exploit development (Track A2).
- **OSEE** (EXP-401) — OffSec's hardest; advanced exploitation. Long-term goal.
- **AIRTP+** or similar — AI red teaming credential (Track B2).

Certs structure learning and pass HR filters — but a public track record (CVEs, bug bounty, writeups) matters more for this field. Pursue both.

ORDER · Suggested master sequence

1. Trunk: C → assembly → RE fundamentals (keep doing web/API pentest in parallel — it's your income/base).
2. Track A1: binary exploitation core (pwn.college + Nightmare). The make-or-break step — see if you LOVE it.
3. Branch B2 in parallel once A1 clicks: LLM red teaming is light and high-demand — quick win + portfolio.
4. Track A2/A3: Windows exploit dev + heap. Add OSCP/CRTP around here for the pentest/red-team side.
5. Pick ONE advanced arena (A4 kernel / A5 browser / A6 mobile) based on what gripped you most.
6. Layer A8 fuzzing/vuln research throughout — it's how you find your first real CVE.
7. Tie it together with AD + endpoint/EDR + C2 for full red-team capability.

Don't do everything at once. Foundation → one specialty deep → add lanes. Breadth comes from years, not from parallel cramming.

✓ · Progress Checklist

- C fundamentals solid
- Assembly — reads disassembly
- RE fundamentals + Ghidra
- Binary exploitation core (stack/ROP/format)
- Heap exploitation
- Windows exploit dev
- One advanced arena (kernel/browser/mobile)
- Fuzzing & first vuln-research attempt
- Offensive malware dev (implants + evasion + C2)
- LLM red teaming (PortSwigger + Gandalf + OWASP)
- AD / red team (GOAD/CRTP)
- Endpoint/EDR + C2
- First public proof: CVE / bug bounty / writeup

Ethics & legality: every technique here is for your own isolated lab, CTF platforms, or systems you have explicit written authorization to test (incl. bug-bounty scope). Finding and proving vulnerabilities is legal and valued; using them on systems you don't own or aren't

authorized for is not. Stay on the authorized side — it's also where the career and the money are.

Links current as of June 2026; if one breaks, search the resource name. This is a strong default map, not a contract — adjust as you discover what you love and as the field shifts.